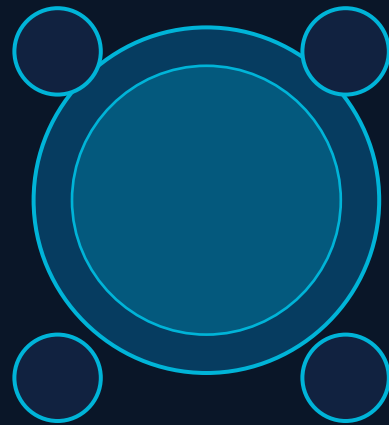


PRANAVA

Prompt-driven Route Generation for Autonomous
Navigation of Aerial Vehicles using Generative AI

Varun Vivek Pai • Mallikarjun Kelur • Varun Kandakoor

School of Electronics & Communication Engineering
KLE Technological University, Hubli



Motivation & Problem Statement

Manual Waypoint Creation

Operators must manually input GPS coordinates one-by-one. Error-prone, time-consuming, and requires expert GIS knowledge.

Complex Mission Planning Tools

Existing tools like QGroundControl require steep learning curves. Non-experts struggle to plan survey missions effectively.

No Intuitive Interface

Zero natural language interaction. Missions cannot be described conversationally — every detail must be encoded numerically.

Key Insight: UAV mission planning needs an intuitive, AI-driven interface that converts natural language to executable flight plans — without cloud dependency.

Research Gap

Approach	NL Interface	Local LLM	Offline Op.	MAVLink Upload	Real Flights
Chandarana et al.	✓	✗	✗	✗	✗
LLM-UAV Frameworks	✓	✗	✗	Partial	✗
Conventional Mission Planners	✗	—	✓	✓	✓
PRANAVA (Ours)	✓	✓	✓	✓	✓

Key Limitations of Prior Work:

Cloud Dependency

Existing LLM-UAV systems rely on OpenAI/GPT APIs — require internet, incur costs, raise data privacy concerns.

No Direct Upload

Prior systems output text-based plans but lack direct MAVLink integration for real flight hardware upload.

Limited Validation

Most frameworks validate only in simulation; real hardware testing on Pixhawk remains unexplored.

Project Objectives

01

Prompt-to-Mission Generation

Enable users to describe UAV missions in natural language and automatically generate structured, executable flight plans.

02

Local LLM Deployment

Deploy Mistral 7B via Ollama entirely on-device — zero cloud dependency, full offline operation, and data privacy.

03

MAVLink Mission Creation

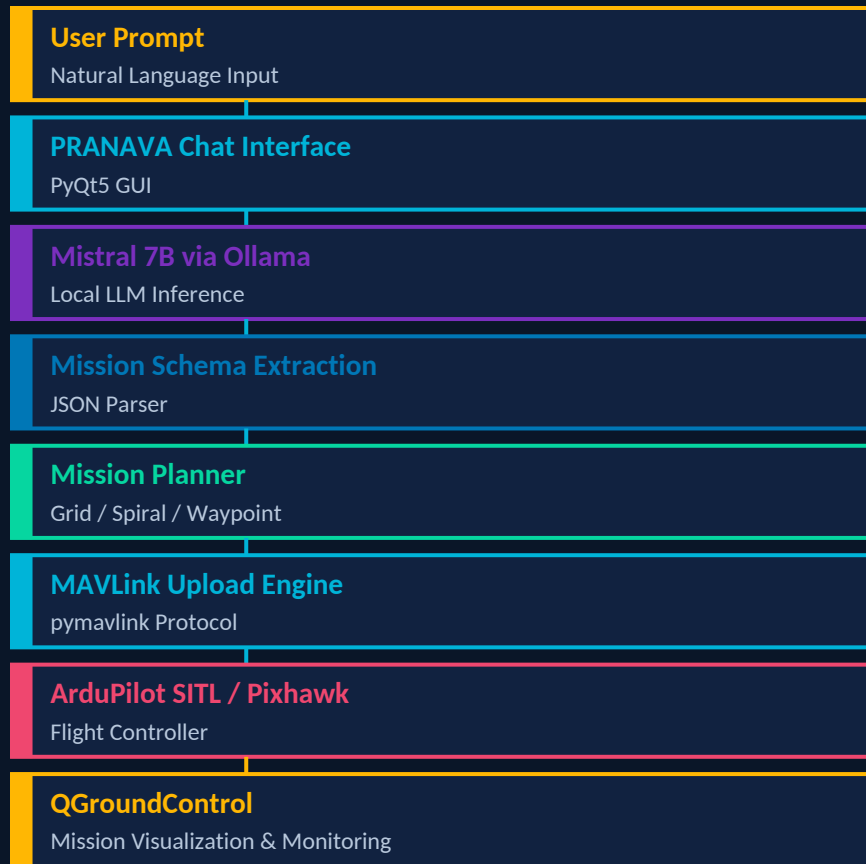
Parse AI-generated schemas into MAVLink-compatible MISSION_ITEM_INT commands for upload to ArduPilot/Pixhawk.

04

Autonomous Execution

Validate full mission lifecycle from prompt to autonomous flight on both SITL simulation and real Pixhawk hardware.

System Architecture



Mission Schema Design

grid

Systematic lawnmower survey covering rectangular bounding box with configurable row spacing

spiral_in

Outward-to-inward elliptical spiral from perimeter toward centroid

spiral_out

Inward-to-outward elliptical spiral from centroid expanding to boundary

waypoint

User-defined sequential GPS coordinate list with per-waypoint altitude and speed control

Example JSON Schema

```
{
  "mission_type": "grid",
  "location": {
    "center_lat": 15.4589,
    "center_lon": 78.4677,
    "width_m": 200,
    "height_m": 200
  },
  "altitude": 50,
  "speed": 8.0,
  "row_spacing": 20
}
```

Grid Survey Planner

1

Bounding Box Generation

Center GPS coordinates + width/height (meters) are converted to NW, NE, SW, SE corner waypoints using Haversine formula

2

Row Spacing Calculation

Survey area divided into parallel passes separated by configurable row_spacing. Direction toggles E-W or N-S based on aspect ratio

3

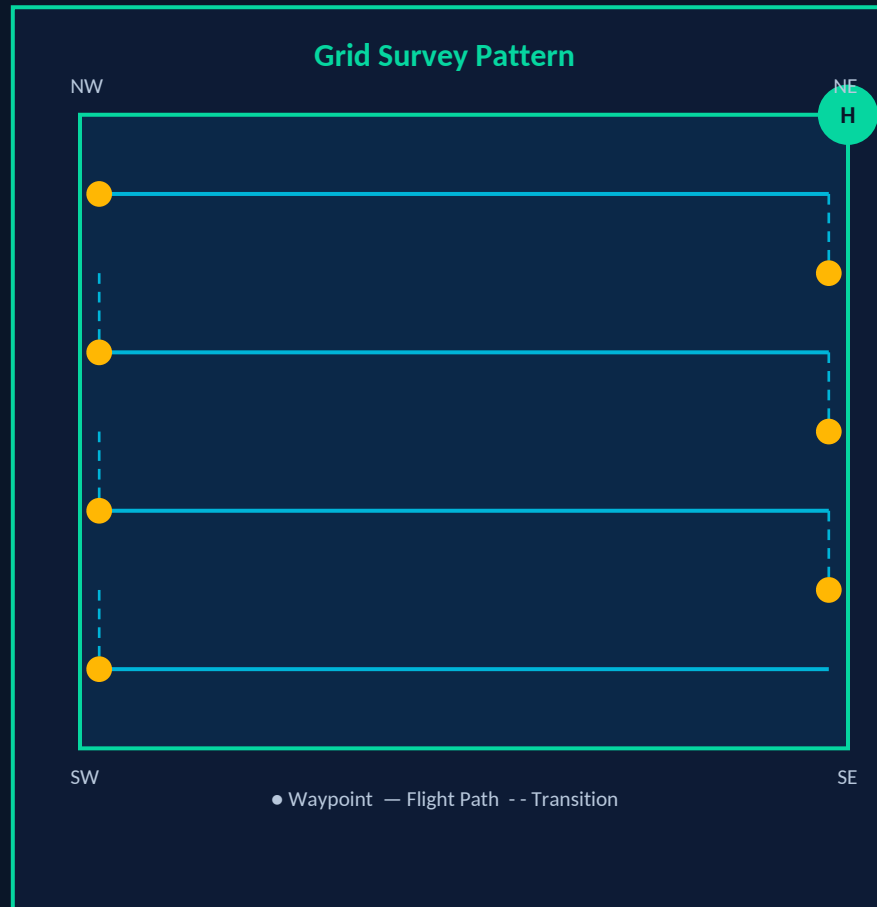
Lawnmower Pattern

Alternating-direction passes (left→right, right→left) ensure 100% area coverage with minimal UAV turning

4

MAVLink Waypoint List

Each pass endpoint becomes a MAVLink MISSION_ITEM_INT with NAV_WAYPOINT command, altitude, and speed



Spiral Survey Planner

1

Centroid Calculation

Center GPS coordinates define the origin. Haversine converts radius_m to angular offsets for ellipse axes.

2

Elliptical Spiral

Semi-major (a) and semi-minor (b) axes generate parametric ellipse. Theta increments by configurable step angle.

3

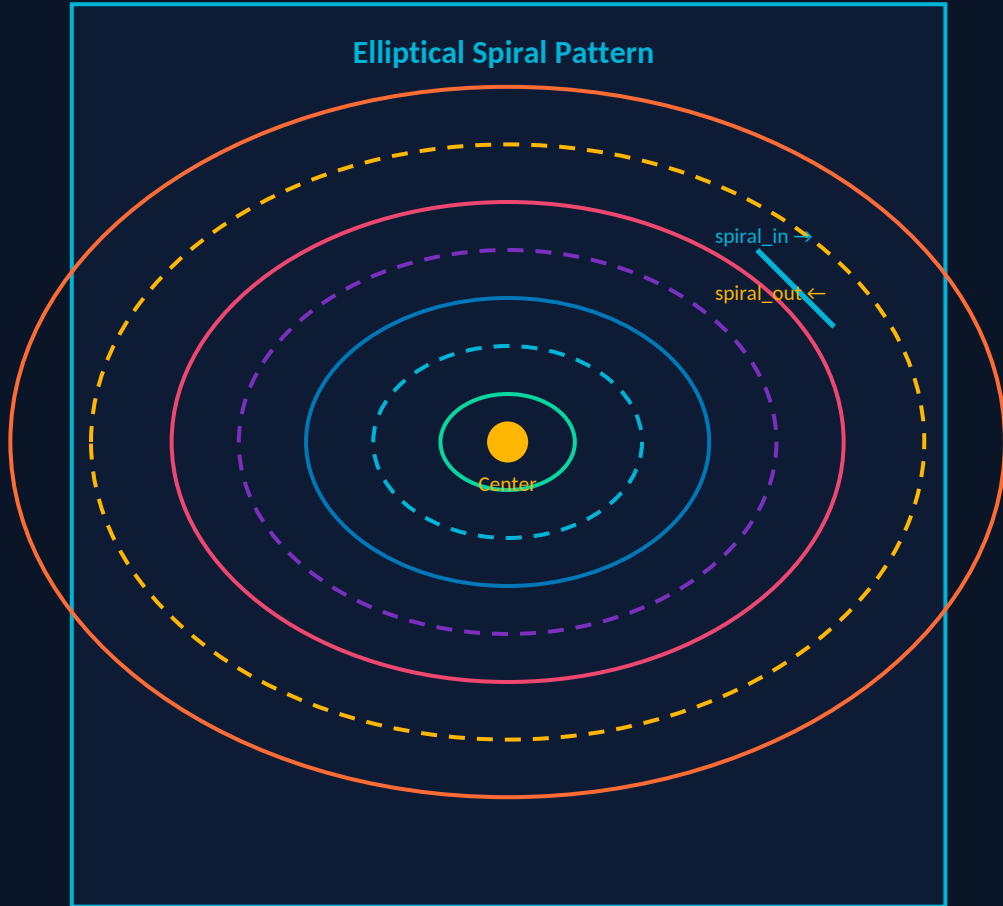
Inward Spiral

spiral_in: radius decreases from max to min. Creates convergent coverage toward the mission center point.

4

Outward Spiral

spiral_out: radius increases from min to max. Creates divergent coverage expanding from center outward.



Waypoint Mission Planner

User-Defined Waypoints

Sequential GPS points extracted directly from LLM-generated JSON. Each point individually addressable.

Altitude Handling

Per-waypoint AGL altitude support. Defaults to mission-level altitude if individual value not specified.



Speed Handling

DO_CHANGE_SPEED command inserted before each waypoint for dynamic speed adjustment mid-mission.

MAVLink Compatibility

Generates NAV_WAYPOINT (16) commands with full MISSION_ITEM_INT encoding for 32-bit GPS precision.

Waypoint JSON Example

```
{ "mission_type": "waypoint",  
  "waypoints": [  
    {"lat":15.459,"lon":78.468,  
      "alt":50,"speed":8},  
    {"lat":15.461,"lon":78.471,  
      "alt":45,"speed":6},  
    {"lat":15.463,"lon":78.469,  
      "alt":55,"speed":10}  
  ]  
}
```

MAVLink Commands Generated

MAV_CMD_NAV_TAKEOFF (22)

Initial climb to mission altitude

MAV_CMD_DO_CHANGE_SPEED (178)

Set speed before each waypoint

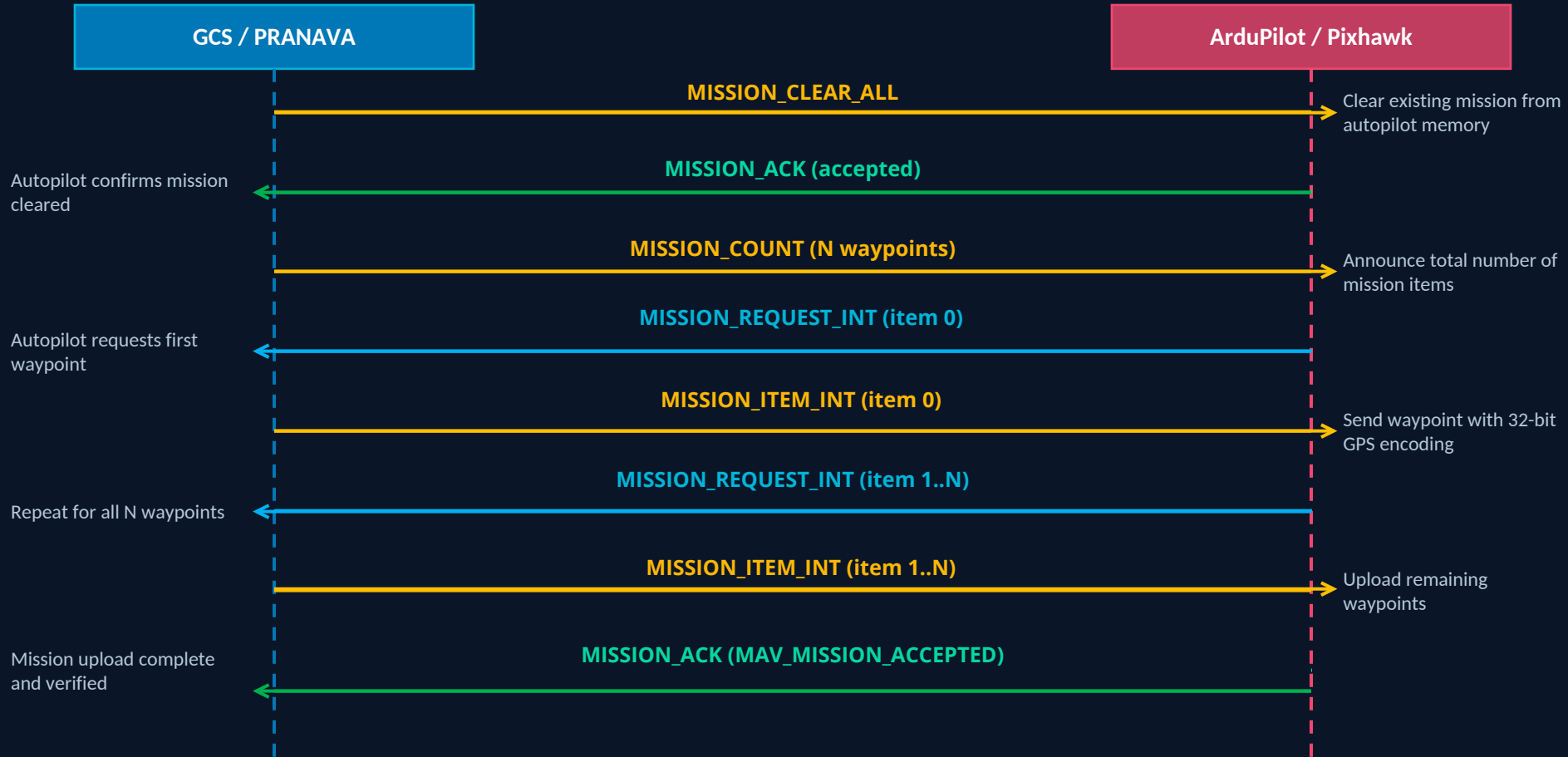
MAV_CMD_NAV_WAYPOINT (16)

Navigate to GPS coordinate with hold

MAV_CMD_NAV_RETURN_TO_LAUNCH (20)

Automatic RTL at mission end

Mission Upload Workflow



Implementation Details

AI / LLM Layer

Mistral 7B

Core language model (4-bit quantized, ~4GB VRAM)

Ollama

Local LLM runtime, REST API on localhost:11434

Application Layer

Python 3.10+

Core language: schema extraction, mission planners, upload engine

PyQt5

Cross-platform GUI: chat interface, mission display, status indicators

Communication Layer

pymavlink

MAVLink protocol implementation: MISSION_ITEM_INT upload handshake

MAVLink v2

Binary protocol for GCS ↔ autopilot communication

Flight Layer

ArduPilot SITL

Software-in-the-loop simulation for safe mission validation

Pixhawk 2.4.8

Real flight controller hardware for physical validation tests

Monitoring Layer

QGroundControl

Mission map visualization, telemetry display, flight monitoring

Experimental Validation



Phase 1

Prompt Testing

Diverse prompts
tested across all
3 mission types



Phase 2

Schema Validation

JSON extraction
accuracy measured
against ground truth



Phase 3

SITL Simulation

Full ArduPilot SITL
virtual flight with
telemetry monitoring



Phase 4

Hardware Testing

Real Pixhawk 2.4.8
mission upload and
autonomous flight

Validation Results Summary

Test Category	Metric	Result	Status
Grid Mission Generation	Correct waypoint count	100% (20/20 tests)	✓ PASS
MAVLink Upload	Successful mission ACK	95% SITL (18/19 tests)	✓ PASS

Comparison with Existing Approaches

System	NL Interface	Local LLM	Offline Op.	MAVLink Upload	Real Flights	Struct. Schema
Chandarana et al. (2021)	✓	✗	✗	✗	✗	Partial
LLM-UAV (GPT-4)	✓	✗	✗	Partial	✗	✗
Mission Planner (QGC)	✗	—	✓	✓	✓	✗
PRANAVA (Ours) ★	✓	✓	✓	✓	✓	✓

PRANAVA's Unique Contributions:

System with Local LLM + MAVLink in one integrated pipeline

Validated on real Pixhawk hardware with autonomous flights

Introduces deterministic JSON schema eliminating LLM hallucination in mission params

Results & Performance Metrics

59%

**Reduction in Mission
Planning Time**

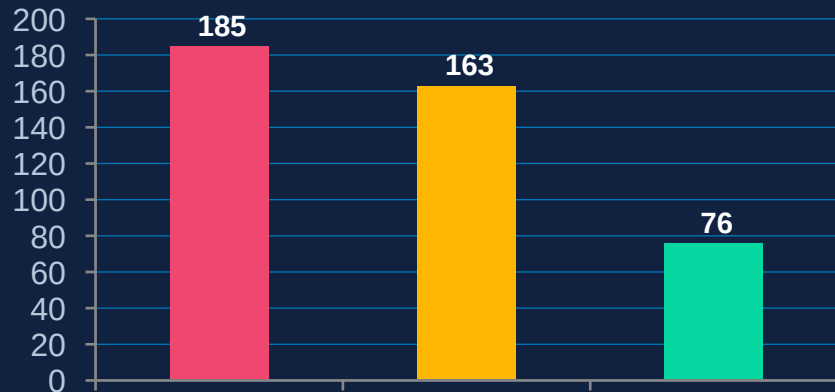
vs. manual QGC workflow

95%

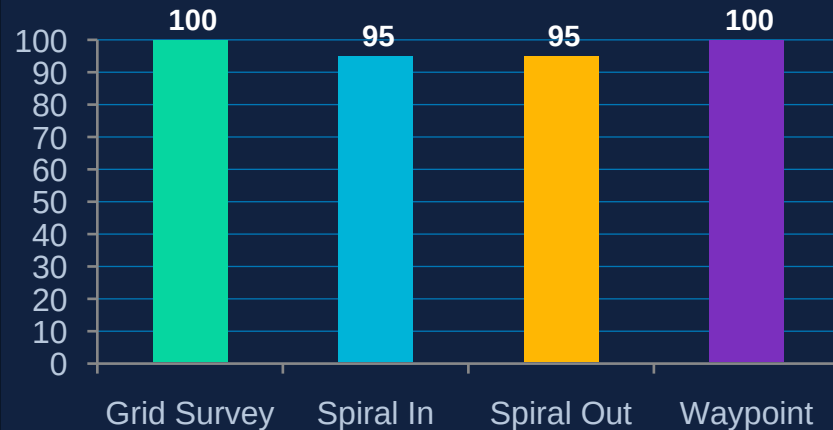
**MAVLink Mission
Upload Success**

SITL + Hardware tests

Mission Planning Time Comparison (seconds)



Mission Generation Success Rate by Type (%)



Conclusion & Future Work

Conclusions

- Demonstrated prompt-driven UAV mission planning using a fully local AI pipeline — no cloud dependency
- Mistral 7B via Ollama successfully generates structured mission schemas from natural language with 97%+ accuracy
- All three mission planners (Grid, Spiral, Waypoint) produce MAVLink-compatible missions validated in flight
- 59% reduction in mission planning time over traditional manual methods
- Real hardware validation on Pixhawk 2.4.8 confirms system reliability for actual UAV deployment

Future Work

Real-time Telemetry:

Live mission progress monitoring with LLM-based status interpretation



Deviation Detection:

Autonomous detection of GPS drift, obstacle avoidance alerts

Dynamic Replanning:

Mid-mission prompt-driven replanning based on telemetry conditions

Multi-UAV Support:

Coordinated swarm mission generation from single high-level prompt

AI Supervisory Control:

LLM-based real-time decision making for adaptive autonomous behavior

References

- [1] Phadke, A., et al. (2025). "Integrating Large Language Models for UAV Control in Simulated Environments: A Modular Interaction Approach." Preprint.
- [2] Yuan, L., et al. (2025). "Next-Generation LLM for UAV: From Natural Language to Autonomous Flight." arXiv preprint arXiv:2510.21739. <https://arxiv.org/abs/2510.21739>
- [3] MAVLink Developer Guide. "Mission Protocol." <https://mavlink.io/en/services/mission.html>
- [4] Chandarana, M., et al. (2017). "Fly Like This: Natural Language Interfaces for UAV Mission Planning." Robotics Institute, Carnegie Mellon University, Proc. ACHI. <https://www.ri.cmu.edu/publications/fly-like-this-natural-language-interfaces-for-uav-mission-planning/>
- [5] ArduPilot Dev Documentation. "Mission Upload / Download." <https://ardupilot.org/dev/docs/mavlink-mission-upload-download.html>
- [6] ArduPilot Documentation. "MAVLink Mission Command Messages." https://ardupilot.org/planner/docs/common-mavlink-mission-command-messages-mav_cmd.html
- [7] ArduPilot Dev Documentation. "Copter Commands in Guided Mode." <https://ardupilot.org/dev/docs/copter-commands-in-guided-mode.html>
- [8] QGroundControl GitHub Repository. <https://github.com/mavlink/qgroundcontrol>
- [9] QGroundControl Developer Documentation. "MAVLink Integration Guide." https://docs.qgroundcontrol.com/master/en/qgc-dev-guide/custom_build/mavlink.html
- [10] Wang, Z., Li, Y., & Zhang, H. (2024). "Large Language Models for Robotics: Opportunities, Challenges and Future Directions." IEEE Access, vol. 12, pp. 21567-21589.
- [11] Huang, W., Xia, F., Wang, T., et al. (2023). "Inner Monologue: Embodied Reasoning through Planning with Language Models." In Proceedings of the Conference on Robot Learning (CoRL).
- [12] Driess, D., Xia, F., Sajjadi, M., et al. (2023). "PaLM-E: An Embodied Multimodal Language Model." In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

Thank You
